

Create a new React Application with the name “myfirstreact”, Run the application to print “welcome to the first session of React” as heading of that page.

1. To create a new React app, Install Nodejs and Npm from the following link:

<https://nodejs.org/en/download/>

2. Install Create-react-app by running the following command in the command prompt:

```
C:>npm install -g create-react-app
```

3. To create a React Application with the name of “myfirstreact”, type the following command:

```
C:>npx create-react-app myfirstreact
```

4. Once the App is created, navigate into the folder of myfirstreact by typing the following command:

```
C:>cd myfirstreact
```

5. Open the folder of myfirstreact in Visual Studio Code
6. Open the App.js file in Src Folder of myfirstreact
7. Remove the current content of "App.js"
8. Replace it with the following:

```
function App() {  
  return (  
    <h1> Welcome the first session of React </h1>  
  );  
}
```

9. Run the following command to execute the React application:

```
C:\myfirstreact>npm start
```

10. Open a new browser window and type "localhost:3000" in the address bar



Create a react app for Student Management Portal named StudentApp and create a component named Home which will display the Message "Welcome to the Home page of Student Management Portal". Create another component named About and display the Message "Welcome to the About page of the Student Management Portal". Create a third component named Contact and display the Message "Welcome to the Contact page of the Student Management Portal". Call all the three components.

1. Create a React project named "StudentApp" type the following command in terminal of Visual studio:

```
C:>npx create-react-app StudentApp
```

2. Create a new folder under Src folder with the name "Components". Add a new file named "Home.js"

3. Type the following code in Home.js

```
import React, {Component} from 'react';

import class Home extends Component{
  render(){
    <div>
      <h3> Welcome to the Home Page of Student Management Portal </h3>
    </div>
  }
}
```

4. Under Src folder add another file named "About.js"
5. Repeat the same steps for Creating "About" and "Contact" component by adding a new file as "About.js", "Contact.js" under "Src" folder and edit the code as mentioned for "Home" Component.
6. Edit the App.js to invoke the Home, About and Contact component as follows:

```
import logo from './logo.svg';
import './App.css';
import {Home} from './Components/Home';
import {About} from './Components/About';
import {Contact} from './Components/Contact';

function App() {
  return (
    <div className="container">
      <Home/>
      <About/>
      <Contact/>
    </div>
  );
}

export default App;
```

7. In command Prompt, navigate into StudentApp and execute the code by typing the following command:

```
C:\studentapp>npm start
```

8. Open browser and type "localhost:3000" in the address bar:



Welcome to the Home Page of Student Management Portal

Welcome to the About Page of Student Management Portal

Welcome to the Contact Page of Student Management Portal

Create a react app for Student Management Portal named scorecalculatorapp and create a function component named "CalculateScore" which will accept Name, School, Total and goal in order to calculate the average score of a student and display the same.

1. Create a React project named "scorecalculatorapp" type the following command in terminal of Visual studio:

```
C:>npx create-react-app scorecalculatorapp
```

2. Create a new folder under Src folder with the name "Components". Add a new file named "CalculateScore.js"
3. Type the following code in CalculateScore.js

```

import '../Stylesheets/mystyle.css'

const percentToDecimal= (decimal) => {
  return (decimal.toFixed(2) + '%')
}

const calcScore = (total, goal) => {
  return percentToDecimal(total/goal)
}

export const CalculateScore = ({Name, School, total, goal}) => (
  <div className="formatstyle">
    <h1><font color="Brown">Student Details:</font></h1>
    <div className="Name">
      <b><span> Name: </span> </b>
      <span>{Name}</span>
    </div>
    <div className="School">
      <b><span> School: </span> </b>
      <span>{School}</span>
    </div>
    <div className="Total">
      <b><span>Total: </span> </b>
      <span>{total}</span>
      <span>Marks</span>
    </div>
    <div className="Score">
      <b>Score:</b>
      <span>
        {calcScore(
          total,
          goal
        )}
      </span>
    </div>
  </div>
)

```

4. Create a Folder named Stylesheets and add a file named "mystyle.css" in order to add some styles to the components:

```

.Name
{
  font-weight:300;
  color:blue;
}
.School
{
  color:crimson;
}
.Total
{
  color:darkmagenta;
}
.formatstyle
{
  text-align:center;
  font-size:large;
}
.Score
{
  color:forestgreen;
}

```

5. Edit the App.js to invoke the CalculateScore functional component as follows:

```

import {CalculateScore} from '../src/components/CalculateScore';

function App()
{

return(
<div>
  <CalculateScore Name={"Steeve"}
    School={"DNV Public School"}
    total={284}
    goal={3}
  />
</div>
)
}

export default App;

```


6. In command Prompt, navigate into scorecalculatorapp and execute the code by typing the following command:

```
C:\scorecalculatorapp>npm start
```

7. Open browser and type "localhost:3000" in the address bar:



Student Details:

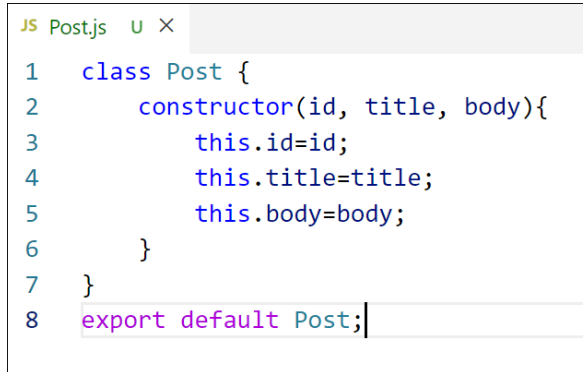
Name: Steeve

School: DNV Public School

Total: 284Marks

Score:94.67%

1. Create a new react application using *create-react-app* tool with the name as “blogapp”
2. Open the application using VS Code
3. Create a new file named as **Post.js** in **src** folder with following properties



```
JS Post.js U X
1  class Post {
2      constructor(id, title, body){
3          this.id=id;
4          this.title=title;
5          this.body=body;
6      }
7  }
8  export default Post;
```

Figure 1: Post class

4. Create a new class based component named as **Posts** inside **Posts.js** file

```

JS Posts.js U X
1 class Posts extends React.Component {
2   constructor(props){
3     super(props);
4   }
5 }

```

Figure 2: Posts Component

5. Initialize the component with a list of Post in state of the component using the constructor
6. Create a new method in component with the name as **loadPosts()** which will be responsible for using Fetch API and assign it to the component state created earlier. To get the posts use the url (<https://jsonplaceholder.typicode.com/posts>)

```

JS Posts.js U X
1 class Posts extends React.Component {
2   constructor(props){
3     super(props);
4     //code
5   }
6   loadPosts() {
7     //code
8   }
9 }

```

Figure 3: loadPosts() method

7. Implement the **componentDidMount()** hook to make calls to **loadPosts()** which will fetch the posts

```

JS Posts.js U X
1 class Posts extends React.Component {
2   constructor(props){
3     super(props);
4     //code
5   }
6   loadPosts() {
7     //code
8   }
9   componentDidMount() {
10    //code
11  }
12 }

```

Figure 4: componentDidMount() hook

8. Implement the **render()** which will display the title and post of posts in html page using heading and paragraphs respectively.

```
JS Posts.js U X
1  class Posts extends React.Component {
2  >    constructor(props) { ...
5      }
6  >    loadPosts() { ...
8      }
9  >    componentDidMount() { ...
11     }
12    render() {
13        //code
14    }
15 }
```

Figure 5: render() method

9. Define a **componentDidCatch()** method which will be responsible for displaying any error happening in the component as alert messages.

```
JS Posts.js U X
1  class Posts extends React.Component {
2  >    constructor(props) { ...
5      }
6  >    loadPosts() { ...
8      }
9  >    componentDidMount() { ...
11     }
12    render() { ...
14    }
15    componentDidCatch(error, info) {
16        //code
17    }
18 }
```

Figure 6: componentDidCatch() hook

10. Add the Posts component to App component.
11. Build and Run the application using *npm start* command.

My Academy team at Cognizant want to create a dashboard containing the details of ongoing and completed cohorts. A react application is created which displays the detail of the cohorts using react component. You are assigned the task of styling these react components.

Download and build the attached react application.



1. Unzip the react application in a folder
2. Open command prompt and switch to the react application folder
3. Restore the node packages using the following commands

```
C:\Windows\System32\cmd.exe  
  
C:\CTS-NewHandsOns\ReactHandsOns\cohortstracker>npm install
```

Figure 1: Restore packages

4. Open the application using VS Code
5. Create a new CSS Module in a file called "CohortDetails.module.css"
6. Define a css class with the name as "box" with following properties
 - Width = 300px;*
 - Display = inline block;*
 - Overall 10px margin*
 - Top and bottom padding as 10px*
 - Left and right padding as 20px*
 - 1 px border in black color*
 - A border radius of 10px*
7. Define a css style for html <dt> element using tag selector. Set the font weight to 500.
8. Open the cohort details component and import the CSS Module
9. Apply the box class to the container div
10. Define the style for <h3> element to use "green" color font when cohort status is "ongoing" and "blue" color in all other scenarios.
11. Final result should look similar to the below image

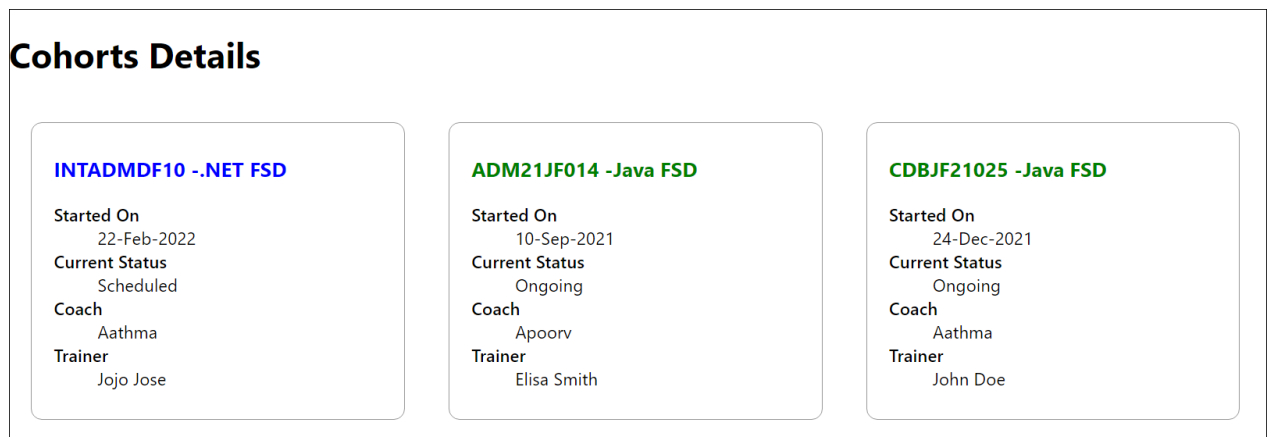


Figure 2: Final Result

1. Create a new React app using *create-react-app* tool with the as “TrainersApp”
2. Open the application using the VS Code
3. Add a new file called *trainer.js* inside the **src folder** and define a class named as “Trainer” with the following properties
 - a. TrainerId
 - b. Name
 - c. Email
 - d. Phone

- e. Technology
- f. Skills

```
JS trainer.js U X
1  class Trainer {
2      constructor(trainerId, name, email, phone, technology, skills) {
3          this.trainerId=trainerId;
4          this.name=name;
5          this.email=email;
6          this.phone=phone;
7          this.technology=technology;
8          this.skills=skills;
9      }
10 }
11 export default Trainer;
```

Figure 1: Trainer.js

4. Create a new TrainersMock.js file which will contain the mock trainer data. Refer the following screenshot for mock data

```
JS trainersmock.js U X
1  import Trainer from "../trainer";
2  const trainersMock = [
3      new Trainer('t-syed8',
4          'Syed Khaleelullah',
5          'khaleelullah@cognizant.com',
6          '97676516962',
7          '.NET',
8          ['C#', 'SQL Server', 'React', '.NET Core']),
9      new Trainer('t-jojo',
10         'Jojo Jose',
11         'jojo@cognizant.com',
12         '9897199231',
13         'Java',
14         ['Java', 'JSP', 'Angular', 'Spring']),
15      new Trainer('t-elisa',
16         'Elisa Jones',
17         'elisa@cognizant.com',
18         '9871212235',
19         'Python',
20         ['Python', 'Django', 'Angular'])
21  ]
22  export default trainersMock;
```

Figure 2: TrainersMock.js

5. Install the support for React router for the dom. Execute the following command.

```
C:\Windows\System32\cmd.exe
C:\CTS-NewHandsOns\ReactHandsOns\trainersapp>npm install react-router-dom@6
```

Figure 3: Install React Router

6. Create new component named as **TrainersList** inside *Trainerlist.js* file. The component should accept the trainer's data as parameter and render it as a list. The list should display names of each trainers which must be clickable like a hyper link. Refer the following screenshot for the component layout.

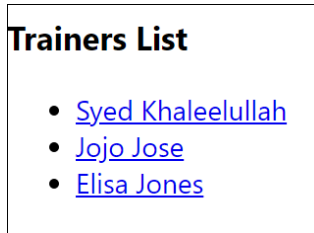


Figure 4: TrainersList Component

7. Create a new component named as Home inside Home.js which will be responsible for displaying the following



Figure 5: Home Component

8. Modify the App component to add support for routing and defining the navigation links to Home component and TrainersList component. Use BrowserRouter, Routes, Route and Link components from the react-router-dom library. Define the following URL
1. / - must redirect to home component
 2. /trainers – must redirect to trainers list component.
- The layout of the page must be similar to the following



Figure 6: App Component

9. Create a new component named **TrainerDetail** in *TrainerDetails.js* file. The component should retrieve a parameter named id from the URL with the help of “useParams” hook from the React router DOM library. It should query the mock trainer data using the id and display the trainer details as show in screenshot. Modify the TrainersList component to add Links to TrainerDetail component while passing the ID. Define a route in App component for the same.

Trainers Details

Syed Khaleelullah (.NET)

khaleelullah@cognizant.com

97676516962

- C#
- SQL Server
- React
- .NET Core

Figure 7: Trainers Detail Component

10. Build and run the application. The complete layout of the application will look as follows.

My Academy Trainers App

[Home](#) | [Show Trainers](#)

Welcome to My Academy trainers page

Figure 8: Home

My Academy Trainers App

[Home](#) | [Show Trainers](#)

Trainers List

- [Syed Khaleelullah](#)
- [Jojo Jose](#)
- [Elisa Jones](#)

Figure 9: Trainers List

My Academy Trainers App

[Home](#) | [Show Trainers](#)

Trainers Details

Syed Khaleelullah (.NET)

khaleelullah@cognizant.com

97676516962

- C#
- SQL Server
- React
- .NET Core

Figure 10: Trainer Details

Create a React Application named “shoppingapp” with a class component named “OnlineShopping” and “Cart”.

1. In Cart class, create 2 properties as mentioned below:
 - Itemname
 - Price

```
{this.props.item.map((item)=>
  {
    return(
      <tr>
        <td> {item.itemname} </td>
        <td> {item.price} </td>
      </tr>
    )})
}
```

2. In OnlineShopping class, create an array of Cart and initialize 5 items.

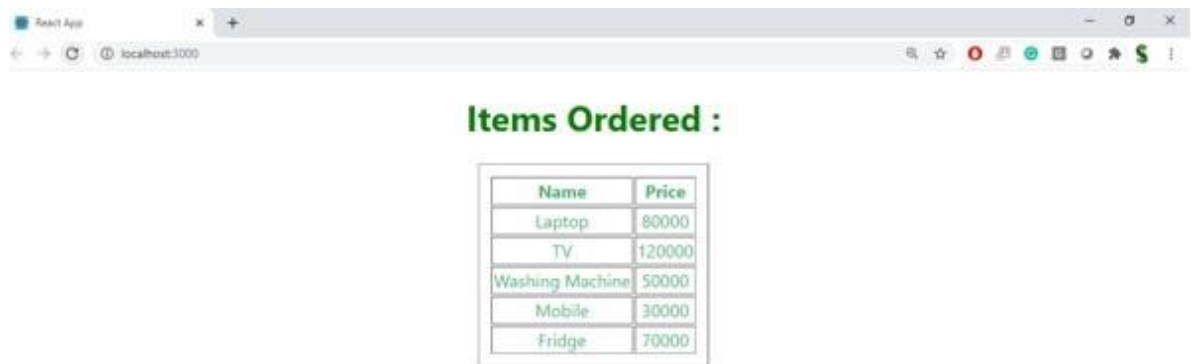
```

export class OnlineShopping extends React.Component {
  render() {
    const CartInfo = [{itemname:"Laptop", price: 80000},
                      {itemname:"TV",price:120000},
                      {itemname:"Washing Machine",price:50000},
                      {itemname:"Mobile",price:30000},
                      {itemname:"Fridge",price:70000}];

    return (
      <div className="mydiv">
        <h1>Items Ordered :</h1>
        <Cart item={CartInfo} />
      </div>
    );
  }
}

```

3. Loop through these items and display the data as shown below:



Name	Price
Laptop	80000
TV	120000
Washing Machine	50000
Mobile	30000
Fridge	70000

Create a React App “counterapp” which will have a component named “CountPeople” which will have 2 methods.

UpdateEntry() ☐ which will display the number of people who entered the mall.

UpdateExit() ☐ which will display the number of people who exited the mall.

Use Constructor and state to Store the entrycount and exitcount.

The component has 2 buttons

1. Login ☐ when clicked, the entrycount should get incremented by 1
2. Exit ☐ when clicked, the exitcount should get incremented by 1

```

constructor() {
  super();
  this.state = {
    entrycount: 0,
    exitcount: 0,
    c: 0
  };
}

updateEntry() {
  this.setState((prevState, props) => {
    return { entrycount: prevState.entrycount + 1 }
  });
}

updateExit()
{
  this.setState((prevState, props) =>
  {
    return { exitcount: prevState.exitcount + 1 }
  })
}

```

The output should be as follows:



Create a React Application named “cricketapp” with the following components:

1. ListofPlayers

- Declare an array with 11 players and store details of their names and scores using the map feature of ES6

```
return(  
  players.map((item)=>  
    {  
      return(  
        <div>  
          <i>Mr. {item.name}<span> {item.score} </span></i>  
        </div>  
      )  
    })  
  )  
)
```


- Filter the players with scores below 70 using arrow functions of ES6.

```
return(
  players.map((item)=>
  {
    if(item.score<=70)
    {
      players70.push(item);
    }
  }
  ),
```

2. IndianPlayers

- Display the Odd Team Player and Even Team players using the Destructuring features of ES6

```
export function OddPlayers([first,, third,,fifth]) {
  return(
    <div>
      <li> First : {first} </li>
      <li> Third : {third} </li>
      <li> Fifth : {fifth}</li>
    </div>))
```

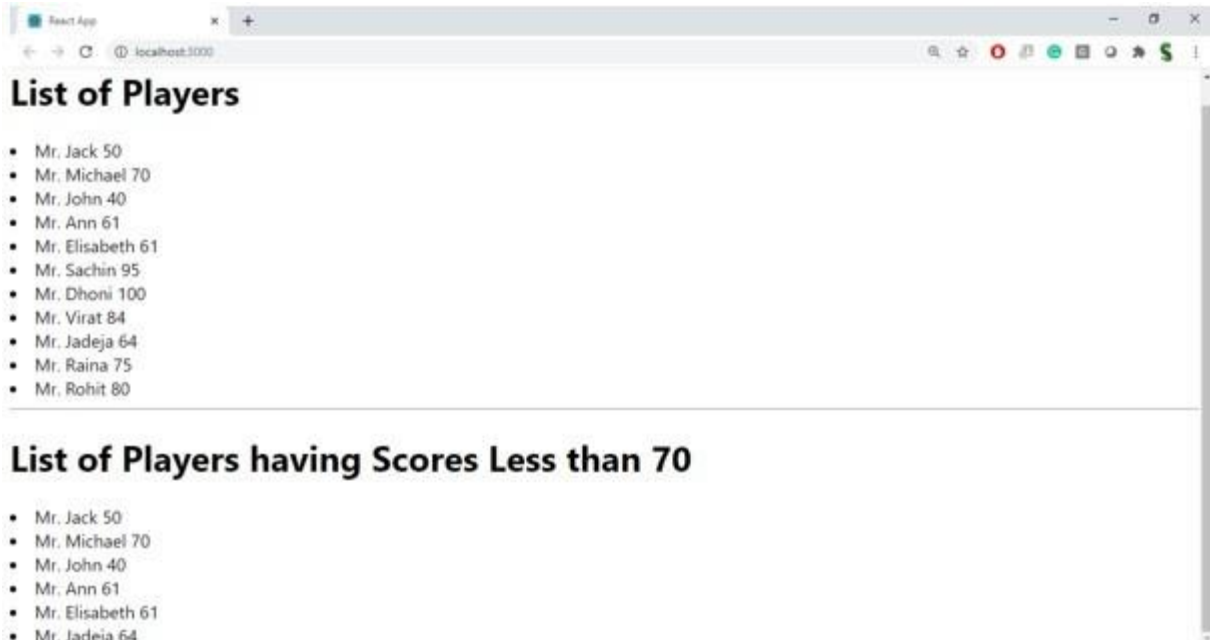
- Declare two arrays T20players and RanjiTrophy players and merge the two arrays and display them using the Merge feature of ES6

```
const T20Players=['First Player','Second Player','Third Player'];
const RanjiTrophyPlayers=['Fourth Player','Fifth Player','Sixth Player'];
export const IndianPlayers=[...T20Players, ...RanjiTrophyPlayers]
```

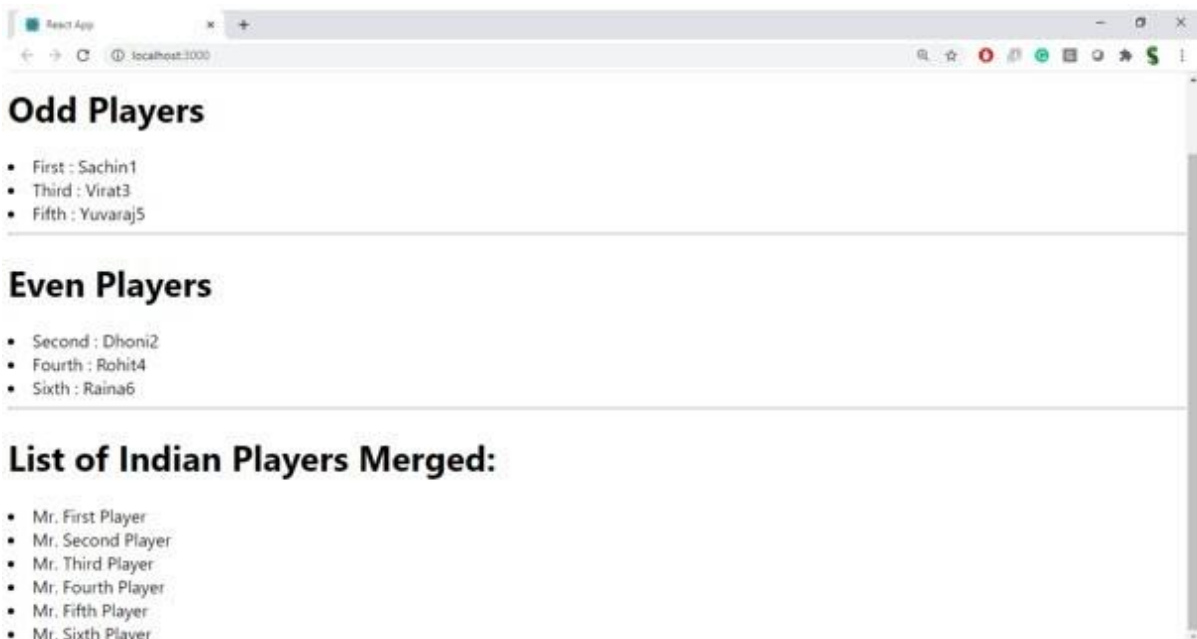
Display these two components in the same home page using a simple if else in the flag variable.

Output:

When Flag=true



When Flag=false



Hint:

```
var flag=true;
if(flag===true)
{
    return(
        <div>
            <h1> List of Players</h1>
            <ListofPlayers players={players}/>
            <hr/>
            <h1> List of Players having Scores Less than 70 </h1>
            <Scorebelow70 players={players}/>
        </div>
    )
}
else{
    return(
        <div>
            <div>
                <h1> Indian Team </h1>
                <h1> Odd Players </h1>
                {OddPlayers(IndianTeam)}
                <hr/>
                <h1> Even Players</h1>
                {EvenPlayers(IndianTeam)}
            </div>
            <hr/>
            <div>
                <h1> List of Indian Players Merged:</h1>
                <ListofIndianPlayers IndianPlayers={IndianPlayers}/>
            </div>
        </div>
    )
}
```

Create a React Application named "officespacerentalapp" which uses React JSX to create elements, attributes and renders DOM to display the page.

Create an element to display the heading of the page.

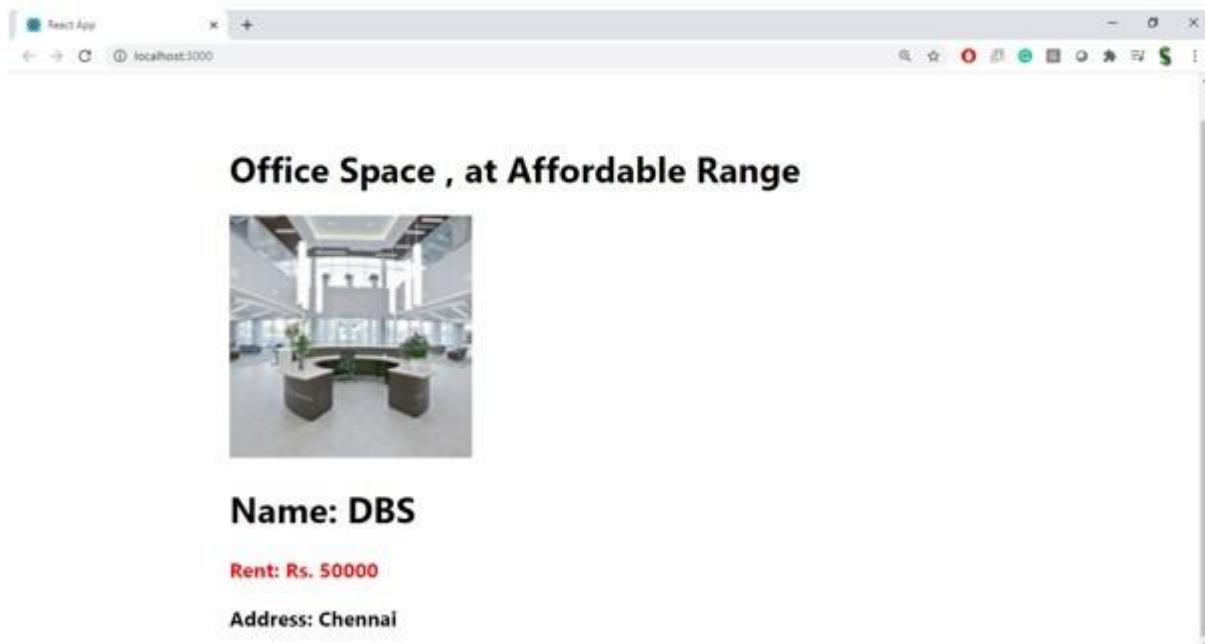
Attribute to display the image of the office space

Create an object of office to display the details like Name, Rent and Address.

Create a list of Object and loop through the office space item to display more data.

To apply Css, Display the color of the Rent in Red if it's below 60000 and in Green if it's above 60000.

Output:



Hint:

```
{
  let colors=[];
  if(ItemName.Rent<=60000)
  {
    colors.push("textRed");
  }
  else{
    colors.push("textGreen");
  }
}

const element="Office Space"
const jsxatt=<img src={sr} width="25%" height="25%" alt="Office Space"/>
const ItemName={Name:"DBS", Rent: 50000, Address:'Chennai'}
<h1>{element} , at Affordable Range </h1>
{jsxatt}
<h1>Name: {ItemName.Name}</h1>
<h3> Rent: Rs. {ItemName.Rent}</h3>
<h3> Address: {ItemName.Address}</h3>
```

Create a React Application “eventexamplesapp” to handle various events of the form elements in HTML.

1. Create “Increment” button to increase the value of the counter and “Decrement” button to decrease the value of the counter. The “Increase” button should invoke multiple methods.
 - a. To increment the value
 - b. Say Hello followed by a static message.



2. Create a button “Say Welcome” which invokes the function which takes “welcome” as an argument.



3. Create a button which invokes synthetic event “OnPress” which display “I was clicked”



Create a “CurrencyConverter” component which will convert the Indian Rupees to Euro when the Convert button is clicked.

Handle the Click event of the button to invoke the handleSubmit event and handle the conversion of the euro to rupees.



Create a React Application named "ticketbookingapp" where the guest user can browse the page where the flight details are displayed whereas the logged in user only can book tickets.

The Login and Logout buttons should accordingly display different pages. Once the user is logged in the User page should be displayed. When the user clicks on Logout, the Guest page should be displayed.



Please sign up.

Login



Welcome back

Logout

Hint:

```
function LoginButton(props) {  
  return (  
    <button onClick={props.onClick}>  
      Login  
    </button>  
  );  
}
```

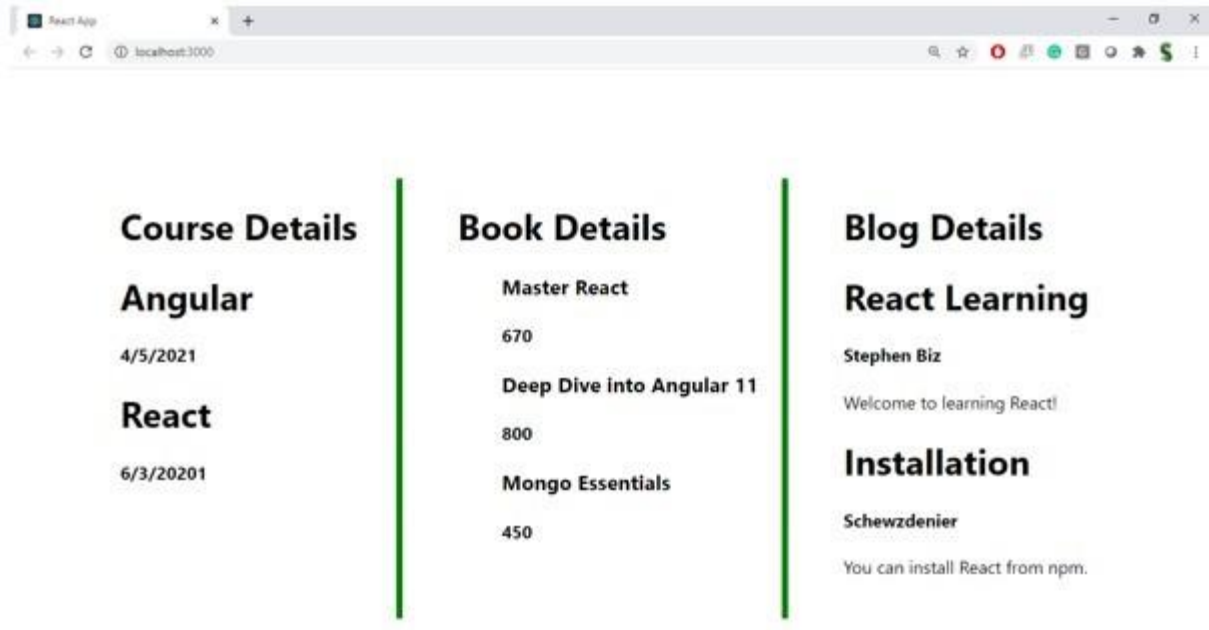
```
function LogoutButton(props) {  
  return (  
    <button onClick={props.onClick}>  
      Logout  
    </button>  
  );  
}
```

```
function Greeting(props) {  
  const isLoggedIn = props.isLoggedIn;  
  if (isLoggedIn) {  
    return <UserGreeting />;  
  }  
  return <GuestGreeting />;  
}
```

Create a React App named "bloggerapp" in with 3 components.

1. Book Details
2. Blog Details
3. Course Details

Implement this with as many ways possible of Conditional Rendering.



Hint:

```
const bookdet = (  
  <ul>  
    {props.books.map((book) =>  
      <div key={book.id}>  
        <h3> {book.bname}</h3>  
        <h4>{book.price}</h4>  
      </div>  
    )}  
  </ul>  
);
```

```
return (
  <div>
    <div>
      <div className="st2">
        <h1> Book Details</h1>
        {bookdet}
      </div>
      <div className="v1">
        <h1> Blog Details</h1>
        {content}
      </div>
      <div className="mystyle1">
        <h1> Course Details</h1>
        {coursedet}
      </div>
    </div>
  </div>
);
}
```

```
export const books=[
  {id:101,bname:'Master React',price:670},
  {id:102,bname:'Deep Dive into Angular 11 ',price:800},
  {id:103,bname:'Mongo Essentials',price:450},
];
```

Developers of Apps Centric Solutions have created an employee management application which supports light and dark themes for the buttons. The current solution uses the react state and props to provide the theme name to be used from App component to Employee List component and from there to Employee Card component. Quality assurance team analyzed the solutions and found the technique being used to be a substandard one. React architect suggested to use the react context API to share the theme name with nested child components instead of passing them down using props from the parent component.

You are assigned the task of converting the application from props only to React Context API.

Application can be downloaded from below



1. Unzip the application and open it using VS Code
2. Go to terminal and execute *npm install* command to restore all the node modules

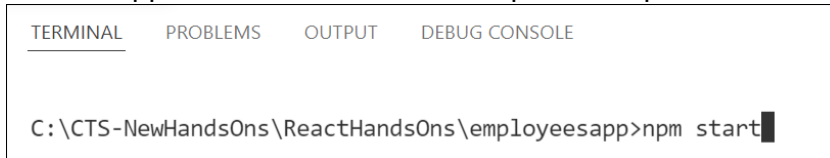


```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE

C:\CTS-NewHandsOns\ReactHandsOns\employeesapp>npm install
```

Figure 1: Restore node modules

3. Run the application once to see the output. Use npm start command.



```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE

C:\CTS-NewHandsOns\ReactHandsOns\employeesapp>npm start
```

Figure 2: Starting application

4. Explore the components present in **App.js**, **EmployeesList.js** and **EmployeeCard.js** files.
5. Create a new file with the name as **ThemeContext.js**. Define a new context in the file with the name as ThemeContext and assign it a default value of 'light' and export it as default from the module.
6. Open App component present in **App.js** file.
 - a. Import the ThemeContext in App component.
 - b. Define the theme context provider to be the entire JSX of the App component.
 - c. Assign the value for the theme provider from the state of the component.
 - d. Modify the call to EmployeeList component so that theme name is no longer passed as props.
7. Go to EmployeeList component present in **EmployeeList.js** file and modify it so that theme name is not passed explicitly to its child component.
8. Go to **EmployeeCard** component inside **EmployeeCard.js** file
 - a. Import the ThemeContext into the component file
 - b. Retrieve the value of the context with the help of **useContext()** and store it in a variable
 - c. Use the variable to pass the className for the buttons.

Create a React App named "ticketraisingapp" which will help to raise a complaint and get it resolved.

Create a component named "ComplaintRegister" with a form containing a textbox to enter the employee name and a textarea to enter the complaint. Use "handleSubmit" event of the button to submit the complaint and generate a Reference number for further follow ups in the alert box.

Output Expected:



Hint:

```
handleChange(event) {  
  this.setState({ [event.target.name] : event.target.value});  
}
```

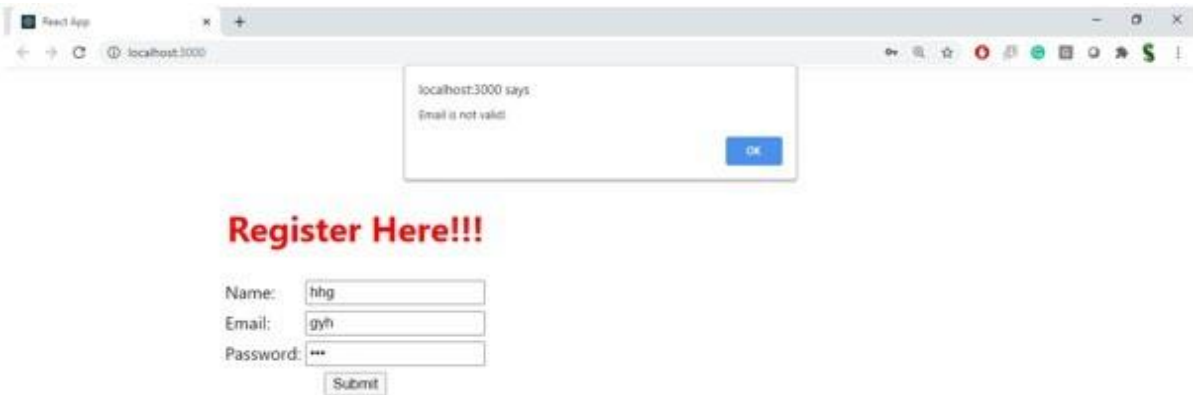
```
handleSubmit(event) {  
  var msg='Thanks ' + this.state.ename + ' \n Your Complaint was Submitted  
ID is: ' + this.state.NumberHolder;  
  alert(msg);  
  event.preventDefault();  
}
```

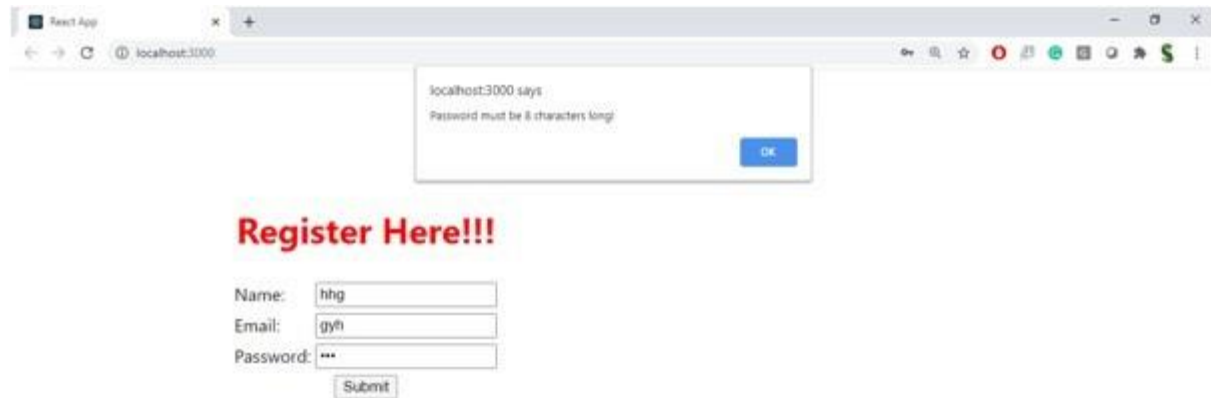
Create a React App named "mailregisterapp" which will have a component named "register.js". Create a form which accepts the name, email and password and validate the fields as per the following:

1. Name should have atleast 5 characters
2. Email should have @ and .
3. Password should have atleast 8 characters.

Ensure that validations are implemented through eventhandle and eventsubmit of a form.

Output Expected:





Hint:

```
handleSubmit = (event) => {  
  event.preventDefault();  
  if(validateForm(this.state.errors)) {  
    alert('Valid Form')  
  }else{  
    if(this.state.errors.fullName!="")  
      {alert(this.state.errors.fullName)}  
    if(this.state.errors.email != "")  
      {alert(this.state.errors.email)}  
    if(this.state.errors.password != "")  
      {alert(this.state.errors.password)}  
  }  
}
```

```
switch (name) {  
  case 'fullName':  
    errors.fullName =  
      value.length < 5  
      ? 'Full Name must be 5 characters long!'  
      : '';  
    break;
```

```
case 'email':
  const validEmailRegex =
    RegExp(/^([^\<>()\[\]\\.,;:\s@"]+(\.[^^\<>()\[\]\\.,;:\s@"]+)*)|(".+")@([^\<>()\[\]\\.,;:\s@"]+\.)+([^\<>()\[\]\\.,;:\s@"]{2,})$/i);

  errors.email =
    validEmailRegex.test(value)
      ? ""
      : 'Email is not valid!';
  break;
```

Create a React Application “fetchuserapp” which will retrieve the user details from <https://api.randomuser.me/> and display the title, firstname and image of a user.

Create a component named “Getuser” and in the asynchronous method “ComponentDidMount ()” invoke the URL using fetch method and the response can be displayed in the render method of the component.

Code Snippet in Getuser Component:

```
async componentDidMount()
{
  const url="https://api.randomuser.me/";
  const response=await fetch(url);
  const data=await response.json();
  this.setState({person:data.results[0],loading:false});
  console.log(data.results[0]);
}
```

Expected Output:



Mr Donato Nunes



My Academy team at Cognizant want to create a dashboard containing the details of ongoing and completed cohorts. A react application is created which displays the detail of the cohorts using React component. You are assigned the task of unit testing the component to ensure it's free of bugs.

Download the following attachment to find the application code.



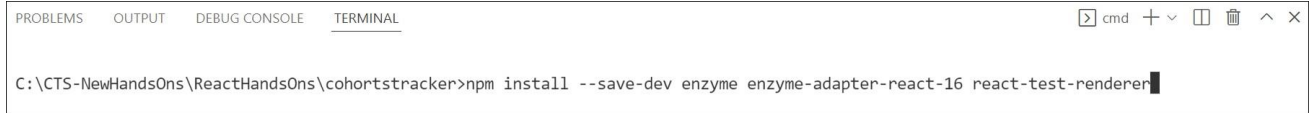
1. Unzip the attachment and open it in visual studio code.
2. Open the terminal and execute the following command to restore the node modules

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

C:\CTS-NewHandsOns\ReactHandsOns\cohortstracker>npm install
```

Figure 1: Packages Restore

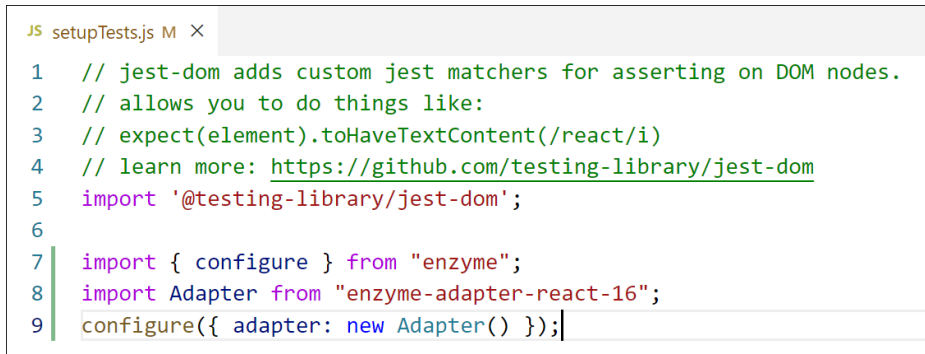
- Execute the following commands to install **Enzyme** and **Enzyme test adapter**



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
C:\CTS-NewHandsOns\ReactHandsOns\cohortstracker>npm install --save-dev enzyme enzyme-adapter-react-16 react-test-renderer
```

Figure 2: Installing Enzyme support

- Go to file explorer and modify the **setupTests.js** file to make it look like the following



```
JS setupTests.js M X
1 // jest-dom adds custom jest matchers for asserting on DOM nodes.
2 // allows you to do things like:
3 // expect(element).toHaveTextContent(/react/i)
4 // learn more: https://github.com/testing-library/jest-dom
5 import '@testing-library/jest-dom';
6
7 import { configure } from "enzyme";
8 import Adapter from "enzyme-adapter-react-16";
9 configure({ adapter: new Adapter() });
```

Figure 3: Configuring Enzyme support

- Add a new unit test file with the name as **CohortDetails.test.js** to write the unit tests for the CohortDetails component defined inside **CohortDetails.js** file.
- Import the following into the unit test
 - mount, shallow from Enzyme library
 - CohortDetails component
 - CohortData array from Cohort.js file
- Define a test suite using **describe()** and name it as "Cohort Details Component"
- Define the following unit tests using **test()**

Table 1: Unit Test - 1

Test - 1	Name: should create the component
A unit test which will load the CohortDetails component in isolation	

Table 2: Unit Test - 2

Test - 2	Name: should initialize the props
This test should mount the component and assign a cohort object to the props. Using matchers ensure that the props are assigned a given cohort object	

Table 3: Unit Test - 3

Test - 3	Name: should display cohort code in h3
----------	--

This test should mount the component and search for an h3 element using find() and verify that it displays the proper cohort code of the object passed in props

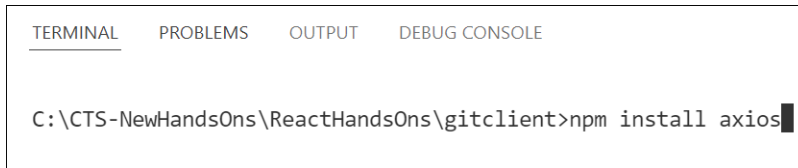
Table 4: Unit Test - 4

Test - 4	Name: should always render same html
This test should be responsible for testing the snapshot of the component.	

9. Build and run the tests using *npm test* command.

As an intern at OpenAI you are assigned the task of creating and testing a React application which will fetch and display a list of repository names for a given user.

1. Create a new React application using *create-react-app* tool and name it as “gitclientapp”.
2. Open the application using VS Code
3. Go to terminal in VS Code and install the support for “axios” react library to make calls to the GitHub API.

A screenshot of the Visual Studio Code interface. The 'TERMINAL' tab is selected at the top. The terminal window shows the command 'C:\CTS-NewHandsOns\ReactHandsOns\gitclient>npm install axios' with a cursor at the end of the line.

```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE  
  
C:\CTS-NewHandsOns\ReactHandsOns\gitclient>npm install axios
```

Figure 1: Install axios library

4. Create a new file with the name as **GitClient.js** in **src folder** of the application
5. Add the following code to create a class named **GitClient** which will make calls to the *api.github.com* to fetch the repositories as follows.

```

JS GitClient.js U X
1 import axios from "axios";
2 class GitClient {
3   static getRepositories(userName) {
4     const url = `https://api.github.com/users/${userName}/repos`;
5     return axios.get(url);
6   }
7 }
8 export default GitClient;

```

Figure 2: GitClient module

6. Modify the App component to use the declared module to fetch and display the repositories data as follows.

```

JS App.js M X
1 import './App.css';
2 import GitClient from './GitClient';
3 import { useEffect, useState } from 'react';
4 function App() {
5   const [repositories, setRepositories] = useState([]);
6   useEffect(() => {
7     GitClient.getRepositories('techiesyed').then(r => setRepositories(r.data));
8   }, [])
9
10  return (
11    <div className="App">
12      <h1>Git repositories of User - TechieSyed</h1>
13      {repositories.map(r =>
14        <p key={r.name}>{r.name}</p>
15      )}
16    </div>
17  );
18 }
19 export default App;

```

Figure 3: App Component

7. Build and Run the application using `npm start` command. The output should look similar to below.

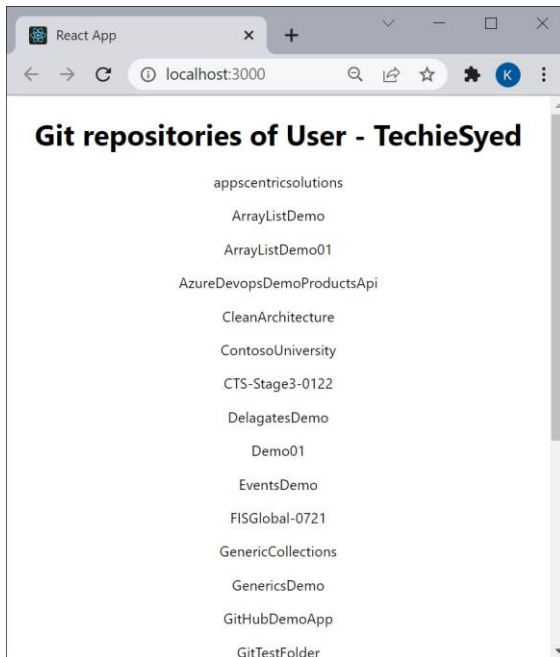


Figure 4: Application Output

8. Create a new unit test file named as **GitClient.test.js** to unit test the newly created module.
9. Import **axios** and **GitClient** into the unit test file.
10. Describe the test name as “Git Client Tests”
11. Create a unit test using the **test()** and give the test name as “should return repository names for techiesyed”
12. Mock the axios object to return the dummy data
13. Invoke the **getRepositories()** method of **GitClient** and see it's returning the mocked data instead of making an actual call to *api.github.com*
14. Run tests using *npm test* command.

```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE

Microsoft Windows [Version 10.0.19041.1415]
(c) Microsoft Corporation. All rights reserved.

C:\CTS-NewHandsOns\ReactHandsOns\gitclient>npm test
```

Figure 5: Run tests